

Vetsyn: An R package for veterinary syndromic surveillance



F.C. Dórea*, S. Widgren, A. Lindberg

Swedish Zoonosis Centre, Department of Disease Control and Epidemiology, National Veterinary Institute (SVA), SE-751 89 Uppsala, Sweden

ARTICLE INFO

Article history:

Received 20 April 2015

Received in revised form

17 September 2015

Accepted 3 October 2015

Keywords:

Animal health

Syndromic surveillance

R programming

ABSTRACT

A recent inventory showed that several countries are exploring varied types of animal health sources for the development of veterinary syndromic surveillance (VSS), however, few systems have documented success after the phase of development or exploratory analysis of data. We suggest there are three main challenges in the current development of animal syndromic surveillance: (i) the lack of standards in disease recording and classification; (ii) the development of statistical methods appropriate to deal with animal data; and (iii) the creation of ready-to-use tools that employ these statistical methods.

We address the latter two challenges and present an R package – *vetsyn* – which covers the steps of VSS implementation from classified data to interface. Detailed tutorials are included with the package. The goal is to provide ready-to-use codes to automatize the process of converting pre-classified animal health data into epidemiological information. The package functions are illustrated using real data and simulated outbreaks. Functions to monitor data daily and weekly are available. The main innovation offered by the package is ability to manage data streams, analyses, alarms and user interface in a continuous flow. We expect that this will facilitate the implementation of syndromic surveillance systems by veterinary epidemiologists and surveillance practitioners.

© 2015 Elsevier B.V. All rights reserved.

1. Introduction

The field of syndromic surveillance has shown rapid growth in public health since 2001. The emergence of the field is attributed to the need for preparedness against disease emergence and bioterrorism threats. This need has been met by advances in infectious disease informatics, which Zeng et al., 2005 has defined as “the science and technologies needed for collecting, sharing, reporting, analyzing, and visualizing infectious disease data and for providing data and decision-making support for infectious disease”. Various documented individual implementations of syndromic surveillance systems, making use of public health data, can be found in the United States (Bravata et al., 2004) and Europe (Dupuy et al., 2013). Moreover, the technologies used in these systems are becoming increasingly open and accessible to surveillance designers interested in implementing syndromic surveillance (Bradley et al., 2005; CDC, 2013). Guidelines for the choice of algorithms (Buckeridge et al., 2005), as well as the overall set up of syndromic surveillance systems (Fouillet et al., 2013) are available, particularly in human health.

In animal health a recent inventory showed that several countries are exploring varied types of animal health sources for the development of veterinary syndromic surveillance (VSS) (Dupuy et al., 2013). However, few systems have documented success after the phase of development or exploratory analysis of data. We suggest there are three main challenges in the current development of animal syndromic surveillance: (i) the lack of standards in disease recording and classification (Dórea et al., 2011; Dórea et al., 2013a); (ii) the development of statistical methods appropriate to deal with animal data (Dórea et al., 2013b); and (iii) the creation of ready-to-use tools that employ these statistical methods.

The development of standards for animal disease classification is a complex issue, which remains in its infancy. Data classification into syndromes remains, for the moment, a task that must be designed and implemented individually by each designer of VSS systems. Assuming that the user has access to a source of animal health data already classified into cases and syndromes, we present an R package that will cover the remaining steps of VSS implementation, as shown in Fig. 1. Detailed tutorials are included with the package. The goal is to provide ready-to-use codes to automatize the process of converting pre-classified animal health data into epidemiological information. In this paper we aim to summarize the current state-of-the-art in veterinary syndromic surveillance, demonstrating that the steps from data to alarms are well streamlined and freely available through the *vetsyn* package. The package

* Corresponding author. Fax: +46 18 674445.

E-mail address: fernanda.dorea@sva.se (A. Lindberg).

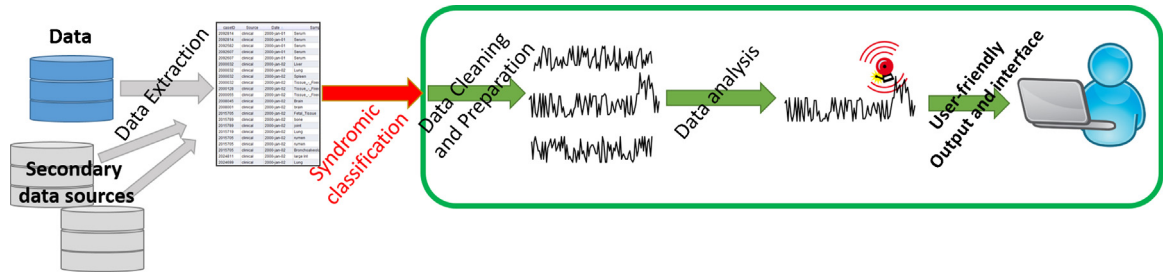


Fig. 1. Steps to implement a syndromic surveillance system, from data extraction to user interface. The R package *vetsyn* offers automated functions to perform the steps within the green polygon, assuming the user has data already classified into syndromes.

functions are illustrated using real data and simulated outbreaks. Functions to monitor data daily and weekly are available.

2. Methods

2.1. Data

The package is loaded with an example dataset of laboratory test requests. In order to comply with data privacy agreements with the institution of origin of those data, for the dataset loaded with the package, it is not informed which animal species the data refer to, nor which is the country of origin of the data. This was necessary because the user of the package will be handling the individual records directly. Detailed analyses of those data, using the package functions, are already publicly available in a tutorial included with the package (Available at: <https://github.com/nandadorea/vetsyn/wiki>).

We have performed analyses, using the *vetsyn* package, in an additional dataset (different from that provided with the package). The dataset used in the analyses reported here is also based on laboratory test requests, and available through the Swedish National Veterinary Institute (SVA). SVA is an agency under the Ministry of Rural Affairs and is the largest and only official laboratory in veterinary medicine in the country. Laboratory test requests for diagnostic of diseases in animals are digitalized daily. As part of the development of a syndromic surveillance system, a classifier based on text mining has been implemented which is applied to the data daily. All laboratory tests not submitted as part of active surveillance or research (passive submissions) are classified into syndromic groups for monitoring (Dórea et al., 2014). To allow early detection, laboratory tests are classified in the day they are received (before tests are available) based on the information available: diagnostic tests requested by the veterinarian and specimen submitted. Further details are provided in Dórea et al. (2013a, 2014).

A subset composed of laboratory test requests for diagnostic in dogs, between January 1st 2011 and December 31st 2013, and classified into two syndromes was chosen: gastrointestinal bacterial diseases (*GIT-Bacterial*); and *Systemic*.

These syndromes were chosen because their time series of daily counts represent varied intensity of temporal effects (very low to very high seasonal effects), as show in Fig. 2.

The package requires, as a minimum dataset format for each event, the syndrome, date of occurrence, and a unique ID. In the dataset evaluated here this unique ID was the animal ID, and the structure is exemplified in Table 1. Any other types of animal health data in which events have been classified into syndromes are suitable for the package. Observations in the dataset do not need to be unique, as long as a unique identifier is present that can allow aggregation of the unique events by date. The package can also deal with multiple levels of aggregation for instance animal ID within a farm ID. In our dataset events are recorded daily. The package allows counts to be aggregated daily or weekly. Users with data

recorded as number of events per week can also take advantage of the package, as long as the weeks are recorded (or transformed) in(to) the ISOweek format (for instance 2014-W01-1 for the first week of the year 2014). The *GIT-Bacterial* syndrome was used to demonstrate the methods available in the package for time series aggregated as weekly event counts (Fig. 2).

2.2. Simulated outbreak signals

No true outbreaks signals related to the monitored syndromes in dogs have been detected in Sweden since the implementation of the syndromic surveillance system. We have therefore injected artificial outbreak signals into the time series used. Evaluating the performance of each detection algorithm in a varied number of outbreak scenarios is beyond the goals of this paper. Instead we have focused on one scenario for outbreak signal, and explored the various detection algorithms available in the package.

Following the methods described in (Dórea et al., 2013c), in the daily time series of *GIT-Bacterial* test requests a log-normal outbreak signal was injected in the last year of data available (starting on September 6th, 2013, the 700th time point of the series). A distance of 15 days from beginning to peak was used to model the outbreak signal, and the magnitude at the peak was three times that of the normal series. The time series with the injected outbreak signal was also aggregated weekly.

For the systemic time series, with strong seasonal patterns, two outbreak signals with the same characteristics were simulated, but in one replica of the series the signal was injected during the season of low submissions (January 25th, 2013, the 540th time point), and in another the outbreak was injected at a time of high submissions (September 6th, 2013, the 700th time point of the series).

The resulting series, with the outbreak signals highlighted in red, are shown in Fig. 3.

2.3. Loading the package

The package is available from GitHub (<https://github.com/>), a platform for revision control, sharing and collaboration in the development of source code. Users can load and use the package directly from their R interface (install.github(“nandadorea/vetsyn”)), or sign in to GitHub to actively contribute to the package’s codes. A full tutorial is available in the package wiki (<https://github.com/nandadorea/vetsyn/wiki>).

Table 1
Example dataset of animal health data classified into syndromes for use with the *vetsyn* package.

Date of submission	Syndrome	ID
25/02/2011	Systemic	69598
03/01/2011	Systemic	34565
03/01/2011	GIT-Macroparasites	352
03/01/2011	GIT-Macroparasites	352
07/01/2011	GIT-Bacterial	78644

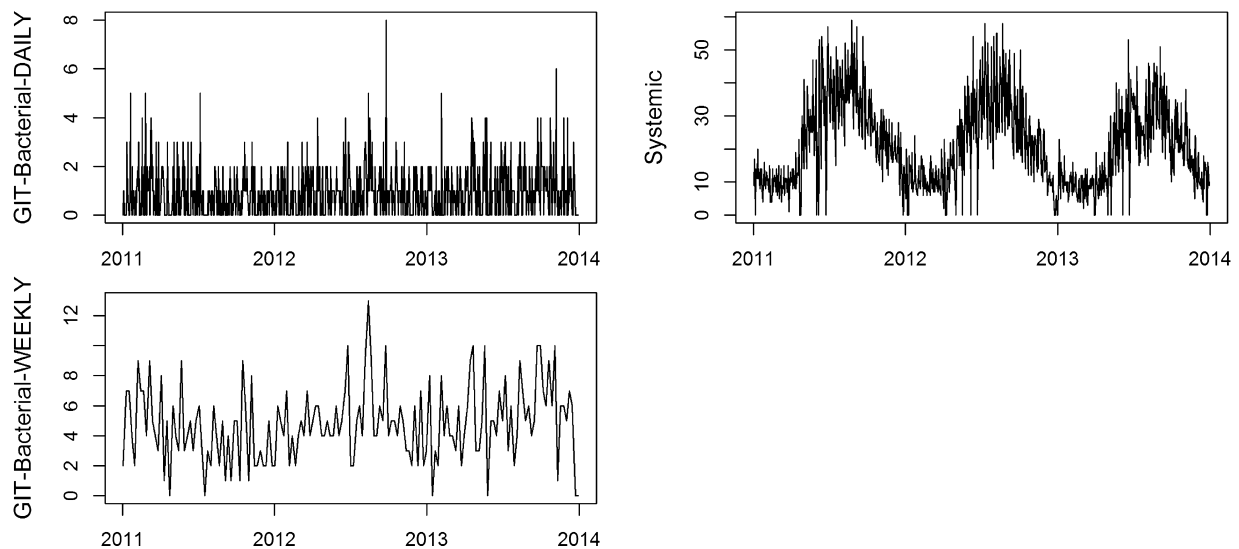


Fig. 2. Time series used to exemplify the methods available in the *vetsyn* package – two syndromes represent the number of laboratory test submissions daily, and counts for gastrointestinal bacterial diseases were also aggregated weekly.

github.com/nandadorea/vetsyn/wiki), and videos demonstrating how to use the package and how to contribute to the codes through GitHub are available at www.sva.se. The package is fully documented through the R help.

2.4. Syndromic: the central S4 class object

Vetsyn makes use of S4 classes, which allow object oriented programming in R. All data relevant for the syndromic surveillance process, from observed data to alarms, are stored in a single object – of class *syndromic*. The operator “@” is used to access different slots of data in the object. When a user provides data in the format shown in Table 1, a single function (*raw.to.syndromic*) will convert those data into a syndromic object with two slots:

- **@observed:** This is a $t \times s$ matrix, where s is the number of syndromic groups found in the data, and t is the number of time

points from the earliest to the most recent date (or week) with events;

- **@dates:** a data frame with the same number of rows as *@observed*, but storing date information for each time point.

A summary of the slots available in a syndromic object is shown in Fig. 4. The user can choose whether to aggregate counts daily (class = *syndromicD*) or weekly (class = *syndromicW*). In the latter case the date information in *@dates* is saved in the ISOweek format (for example 2014-W01-1 for the first week of the year 2014). When creating a syndromic object from data, days or weeks with no observations are assigned a count of zero, and the dates sequence is always completed to a full sequence from the earliest to the latest date found in the dataset (or a specific limit set by the user). Users can also choose which weekdays should be ignored, and specify what to do if counts are found in those days.

For our data, since SVA does not operate on weekends, the function *raw.to.syndromic* was set to remove Saturdays and Sundays,

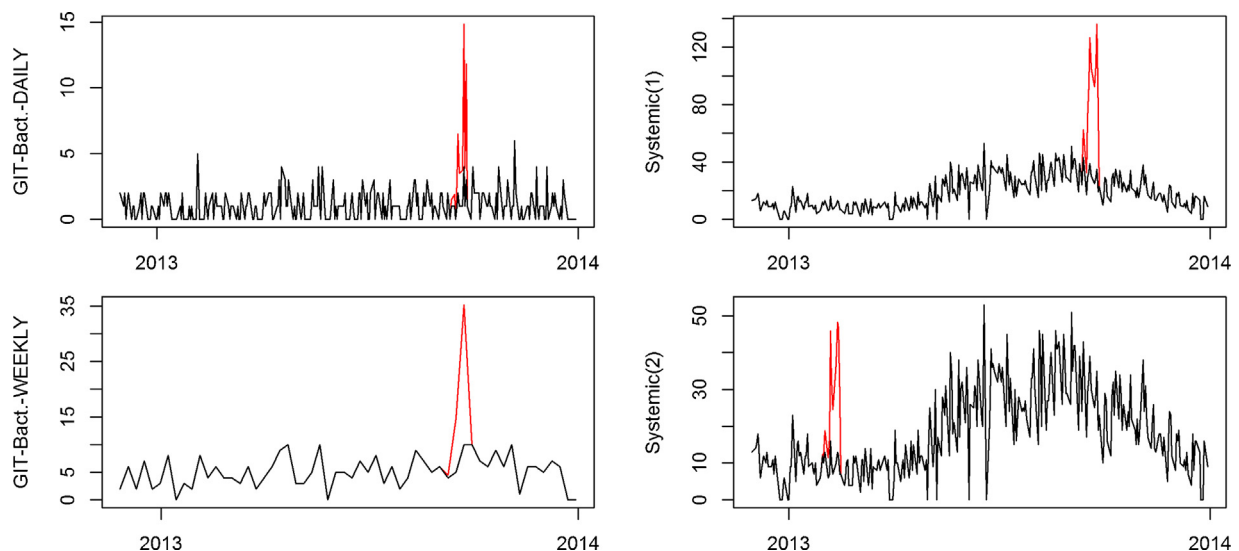


Fig. 3. Time series with injected outbreaks. Outbreaks were modelled as a lognormal curve taking 3 weeks from beginning to peak, and peaking at three times the magnitude of the respective normal counts at the time period. The time series of Bacterial GIT tests is shown daily and weekly, and in the time series of test requests classified as Systemic the outbreak signal was injected in two different seasons.

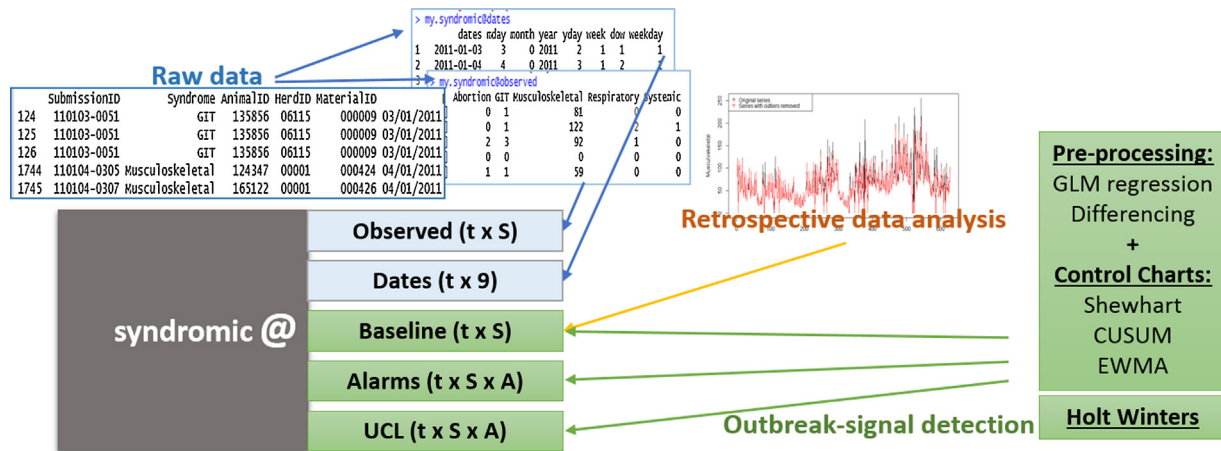


Fig. 4. Schematic structure of a syndromic object, and their links to different steps of the analyses. S = number of syndromes monitored; t = number of time points in the data; A = number of outbreak-signal detection algorithms used.

and assign any events recorded on weekends to the following Monday. Two main syndromic objects were created from the raw data: *dog_synd* and *dog_syndW*, to store the daily and weekly aggregated data respectively. Only the first two years of available data were used in the construction of the initial syndromic objects, in order to simulate implementation at the first day of the third year.

As the analyses are performed, additional slots are added to the object to store the results of syndromic monitoring, as detailed below.

2.5. Retrospective analysis of the data

When setting up syndromic monitoring from available data the first step is a retrospective evaluation of the data, with summary statistics and analysis of temporal effects (Dórea et al., 2013b; Lotze et al., 2008). A single function (*retro_summary*) will generate a markdown file with R codes to perform all basic exploratory analysis detailed in Dórea et al. (2013b):

- daily and weekly time series plots of the data found in the slot *@observed* of the syndromic object provided to the function;
- basic summary statistics, such as mean, quartiles, auto-correlation and partial auto-correlation for week and year;
- box-plots of the data by day-of-week, month and year, intended to allow a preliminary assessment of which temporal effects are present (day-of-week, seasonal effects or trends);
- fitting of a Poisson model to the data using three main formulas (accounting for day-of-week only, day-of-week and month, and day of week and a sinusoidal wave), plus autoregressive components, and the AIC (Akaike Information Criteria) for all models; and
- repetition of the regression models using negative binomial regression.

The markdown file contains a set of R codes to perform such analysis from the data provided. The *knitr* function from the package {knitr} (Xie, 2014) is automatically used to produce an html version with all the results of such analyses. The set of R codes used to generate the html outputs is automatically saved on the users' computer, and can therefore be edited to modify the analyses, add any further evaluations, or simply be reviewed by the user to learn how to set their own R codes.

The function *retro_summary* was applied to the *dog_synd* and *dog_syndW* objects in order to decide the best method to deal with

temporal effects. The resulting html page, with R codes and generated plots, is provided as supplementary material.

2.6. Generating a training baseline for the outbreak signal detection algorithms

The two methods described in Dórea et al. (2013b) for removal of outbreak signals and excessive noise from historical data, in order to produce an outbreak-free baseline to train aberration detection algorithms, were implemented in the package. The parametric method uses the entire data available to fit a regression model, and observations above the 95% confidence interval of the model are substituted by the limit of the interval. In the package, the users can choose any family for a generalized linear model (GLM), and the regression formula is set by the user based on the results of the retrospective evaluation. The independent variables accepted are the common temporal effects in time series monitored in syndromic surveillance (day-of-week, month, sine and cosine, year, and a monotonic trend) and auto-regressive components to account for auto-correlation in the time series.

The non-parametric method is based on moving percentiles. In the package, the user can set the limit percentile and the number of time points to be included in the moving window. A similar correction method is then employed, using the percentile value as the limit for outlier removal.

Once a syndromic object is subjected to the functions *clean_baseline* (parametric) or *clean_baseline_perc* (non-parametric), the data in the slot *@observed* are used to create the outbreak-free baseline, then stored in a slot *@baseline*, which contains the same number of columns as the slot *@observed*, and the same number of time points. Users can apply the methods one column at a time (to customize the methods to each syndrome).

For the analyses presented here, the most suitable method was defined after retrospective analysis. In all cases, the default value in the package for outlier detection (95% percentile or confidence interval) was used, following literature recommendations and previous findings (Dórea et al., 2013b; Tsui et al., 2001).

2.7. Simulating implementation of the system

Once the syndromic objects had observed and baseline data for two years, the implementation of a syndromic surveillance system would be ready to start. It is assumed that from there on new data would become available daily, for which a baseline was

not constructed retrospectively. To simulate this, we have then used the function `update_syndromic` to add an additional one year of data. The outbreak signal detection algorithms implemented in the package only evaluate one time point at a time, as detailed below, and ignore any more recent data. Therefore, it was possible to update the syndromic objects with an entire new year of data at once, and then simulate analysis day by day, as if those data had become available daily. Versions of each syndromic object with and without the simulated outbreak signals injected in this extra year of data (2013) were created.

2.8. Outbreak signal detection

For outbreak signal detection, four detection algorithms were used in the package, namely: Holt-Winters exponential smoothing (Holt-Winters), exponentially weighted moving average control charts (EWMA), cumulative sums control charts (CUSUM) and Shewhart control charts.

These algorithms were chosen based on their previously demonstrated performance with laboratory data (Dórea 2013c,d). A more comprehensive review of these methods, with further references, is provided in Dórea et al., 2013c. In summary, the control-charts use the baseline, assumed to be free of aberrations, to calculate statistics for a stable (in-control) process (Phase I). If the evaluated value falls out of the in-control space (Phase II), an aberration alarm is generated, and the process is considered out-of-control (Woodall, 2006). For the Shewhart control chart this statistic is simply the average of the baseline values, added by a number of standard deviations set by the user (detection limit). The CUSUM and EWMA charts are based in the cumulative sum of deviations from an expected count, over several time points. The exponentially weighted method attributes more weight to observations closer to current value (Woodall, 2006). Explicitly:

$$\text{CUSUM} : C_k = \max\{0, (E_t + \text{Cusum}_{t-1})\} \quad (1)$$

where E_t is the difference between the current observed value and the expected value; and

$$\text{EWMA} : E_t = (1 - \lambda)^t E_0 + \sum_{i=1}^t (1 - \lambda) \lambda I_i \quad (2)$$

where I_t is the individual observation at time t , E_0 is the starting value and λ is the smoothing parameter (>0) that determines the relative weight of current data to past data.

The Holt-Winters is a data-driven smoother that generalizes the exponential smoothing concept (described above for the EWMA control-chart) to express changing trends and cyclic effects as well as the changing level (Elbert and Burkom, 2009).

These algorithms were already available in R. The innovations of the package, which efficiency we aim to demonstrate, are:

- **Iterative application:** The algorithms are applied to a range of time points in an iterative manner, so if syndromic data need to be evaluated for the past 30 days, for instance, the function is called once and the internal loops evaluate one day at a time. This guarantees that the evaluation of a given time point only occurs after all previous time points have been evaluated, and if applicable, any detected outbreak signals have been removed (the outbreak-free baseline has been updated).
- **Detection of deviations:** for the Holt-Winters method, the n -days-ahead forecast of the algorithm is used as an outbreak signal detector – observations above a confidence interval are flagged as “aberrations”. Control charts are already set to provide a detection limit for a given confidence interval.
- **Separating the baseline data from the data being evaluated:** For all algorithms described, the standard implementation in R uses the same time series to learn the baseline values, and then

compare the last observed value with the expectation based on that baseline. Using a *syndromic* object, the training (*@baseline*) and evaluated (*@observed*) data are kept independent. This is important for removal of outbreak signals (see below) while maintaining the original recorded number of events per time point.

- **Recording of the detection limits:** The minimum value that would have generated an alarm in each time point can be recorded in a designated slot (*@UCL*, for upper confidence limit) of the *syndromic* object provided.
- **Outbreak signal removal:** In case an observation is found to be greater than the detection limit chosen, the user can choose to update the outbreak-free baseline (in *@baseline*) by substituting the observed value with this detection limit. The data in *@baseline* therefore will continue to be updated prospectively. In time points with no alarms the number of observed events is also recorded in *@baseline*, but in the time points with alarms the value of the detection limit (minimum number that would have generated an alarm) is recorded instead.
- **Guard-band:** The user can set a guard-band to prevent contamination of the baseline with undetected outbreak signals. That is, the number of recent time points which should not be used from the baseline.
- **Multiple limits:** for each time point evaluated (in each syndrome), if the observed counts are greater than the detection limit the value “1” is recorded in the respective index column (syndrome) and row (time point) of the slot *@alarms*. The user can apply each algorithm with multiple detection limits – that is to say, different confidence intervals. If instead of providing a single value for the detection limit (say 3 standard deviations), the user provides a vector (for example (2.5, 3, 3.5)), then the output saved into *@alarms* is the number of detection limits exceeded by the observed counts (0–3 in this example) – an alarm score. The user can later define the minimum alarm score that should be considered a true alarm. The user can also choose which detection limit to use when recording the upper confidence interval in the slot *@UCL*. The alarm score states clearly which detection limits were met (for instance an alarm score of 2, in the previous example, would allow the user to know that the limits of 2.5 and 3 were exceeded, but not 3.5). However the user can only retrieve the UCL for one of the detection limits. This was necessary in order to keep all slots with the same number of columns and rows, which improves validity checking in the object, and consistency for indexing the arrays.
- **Multiple algorithms:** the user can apply as many algorithms as desired among those available in the package. From the second algorithm on, the outputs of detection (UCL and alarm) are then stored in a third dimension of the corresponding slots. These slots are therefore three dimensional arrays. The final alarm score can be chosen as a sum of the scores from all aberration detection algorithms used.
- **Pre-processing for the control chart methods:** (EWMA, CUSUM and Shewhart): each detection algorithm can be applied directly to the data in the slot *@observed*, or after pre-processing of the data. The user can: set a regression formula to apply any family of generalized linear models to the data, and the detection algorithms are then applied to the residuals of the regression model; choose a window for differencing the data prior to application of the aberration detection algorithms; or choose no pre-processing. Holt-Winters exponential smoothing is always applied directly to the unprocessed data because it can handle temporal effects.

The arguments accepted by each of the detection algorithms are listed in Table 2, which details the original R packages from which each algorithm is loaded, the arguments that are part of the original

Table 2Outbreak-signal detection algorithms implemented in the *vetsyn* package and the parameters available for user customization.

Algorithm	Parameter	Details	Values accepted	Value used ^a
Parameters passed on to the original function from a specific {R package}				
Holt-Winters exponential smoothing {base} ^b	Alpha	Level, trend and	0–1 (def = 0.4)	0.4/--
	Beta	seasonality parameters	0–1 (def = 0)	0/--
	Gamma		0–1 (def = 0.15)	0.15/--
	Seasonal	Method to model the seasonal component	“Additive” (def) or “Multiplicative”	Additive/--
	Nahead	Forecast interval (used as a guard-band)	Integer (def = 7)	5/--
EWMA {qcc} ^b	Frequency	Length of cycles in the data (p.e. week or year)	Integer (def = 7)	5/--
	Lambda	Smoothing coefficient	0–1 (def = 0.2)	0.2/0.2
	CUSUM {qcc}	All parameters modified in <i>vetsyn</i> , as detailed below		--
Shewhart {qcc}	All parameters modified in <i>vetsyn</i> , as detailed below			--
Parameters set in <i>vetsyn</i> for all control-charts				
EWMA, CUSUM and Shewhart	guard.band	Lag between the data being evaluated and the last time point used in the baseline	Integer (def = 7 for daily and 3 for weekly)	5/3
	Pre-process	Whether to pre-process the data and apply the control-charts to the residuals.	FALSE (def), “glm”, “diff”	Syndrome specific ^d
	diff.window	Window of differencing, when that is the method chosen for pre-processing	Integer (def = 7)	5/3
	Family	GLM family, when regression is the method chosen for pre-processing	“poisson” (def) or “nbinom”	“poisson”
	Formula	Regression formula, when regression is the method chosen for pre-processing. For instance “dow + sin + cos” for day-of-the-week and sine/cosine predictors.	Character	Syndrome specific ^d
	Frequency	Length of cycles in the data	Integer (def = 365 for daily and 52 for weekly)	260/52 ^e
Parameters set in <i>vetsyn</i> for all outbreak-signal detection algorithms				
Holt-Winters, EWMA, CUSUM and Shewhart	Syndromes	Which syndromes (columns in @observed) to be evaluated. If not specified, all columns are evaluated.	NULL (def), vector of names, or vector of indexes	NULL
	evaluate.window	Number of recent time points to which outbreak-signal detection should be applied	Integer (def = 1)	260/52 ^e
	baseline.window	Length of the baseline window to be considered, starting from <i>t-guard-band</i>	Integer (def = 365 for daily and 52 for weekly)	520/104
	limit.sd	Limit of detection, in standard deviations ^c	Real value or a vector (def = 2.75, 3, 3.25)	2.75, 3, 3.25
	correct.baseline	Whether to substitute the observed data with the detection limit for days in which alarms are detected, in the slot @baseline. If correction is to be used, the index of the detection limit to be used for correction, from the vector informed in “limit.sd”	FALSE (default) or integer index	TRUE for Holt-Winters, FALSE otherwise
	alarm.dim	The index of the third dimension in the array @alarms into which outputs should be recorded	integer index (def is 1 for Holt-Winters, 2 for EWMA, 3 for Shewhart and 4 for CUSUM)	Holt-Winters = 1; EWMA = 2; Shewhart = 3
	UCL	The index of the detection limit to be recorded, from the vector informed in “limit.sd” (index is 1 when a single limit is used)	FALSE or integer index (def = 1)	2

^a Default value for the argument.^a Value used in the analyses reported, for daily and weekly time series (daily/weekly).^b See Burkow et al. (2007), Dórea et al. (2013c), and Lotze et al. (2008) for more information about the algorithm, and the default parameters set here.^c Rather than a percentage (for instance 95%), the number of standard deviations above the mean is used (for instance 2.5, or 3). This is to ensure comparability with the other detection algorithms used in this package (control charts).^d Described in the results, as the choice depended on retrospective analysis of each series individually.^e One year, since week days only are considered (weekends are excluded).

implementation of the algorithm, and those which were added in *vetsyn* to provide the features listed above.

The functions set in the package *vetsyn* have different defaults depending on whether daily (*syndromicD*) or weekly (*syndromicW*) data are provided. If a syndromic object with daily data is provided, the default baseline window and frequency, for instance, are 365 (one year), and the default guard-band is 7 days (one week). If instead a syndromic object in which counts have been aggregated weekly is provided, then the default values are 52 and 3, respectively (one year and three weeks). The user can of course change the default parameters.

In our analyses we have reproduced the methods demonstrated in (Dórea et al., 2013d), which set detection of outbreak signals based on the parallel application of three algorithms for

time series of daily counts: Holt-Winters (with a week, rather than a year as the frequency of repetition), EWMA and Shewhart. Weekly time series were monitored using only the two control charts. The control charts data are pre-processed according to the method chosen during retrospective evaluation. The settings used in the analyses are also listed in Table 2. Whenever possible the default settings of the package were used, to illustrate the results in a case where the user employs each function with the minimum need for customization. Adjustments were made to reflect the fact that we were using data without weekends. Moreover the pre-processing methods were chosen and set individually, while in the package the default is FALSE to avoid that users employ methods without proper individual analyses of their data.

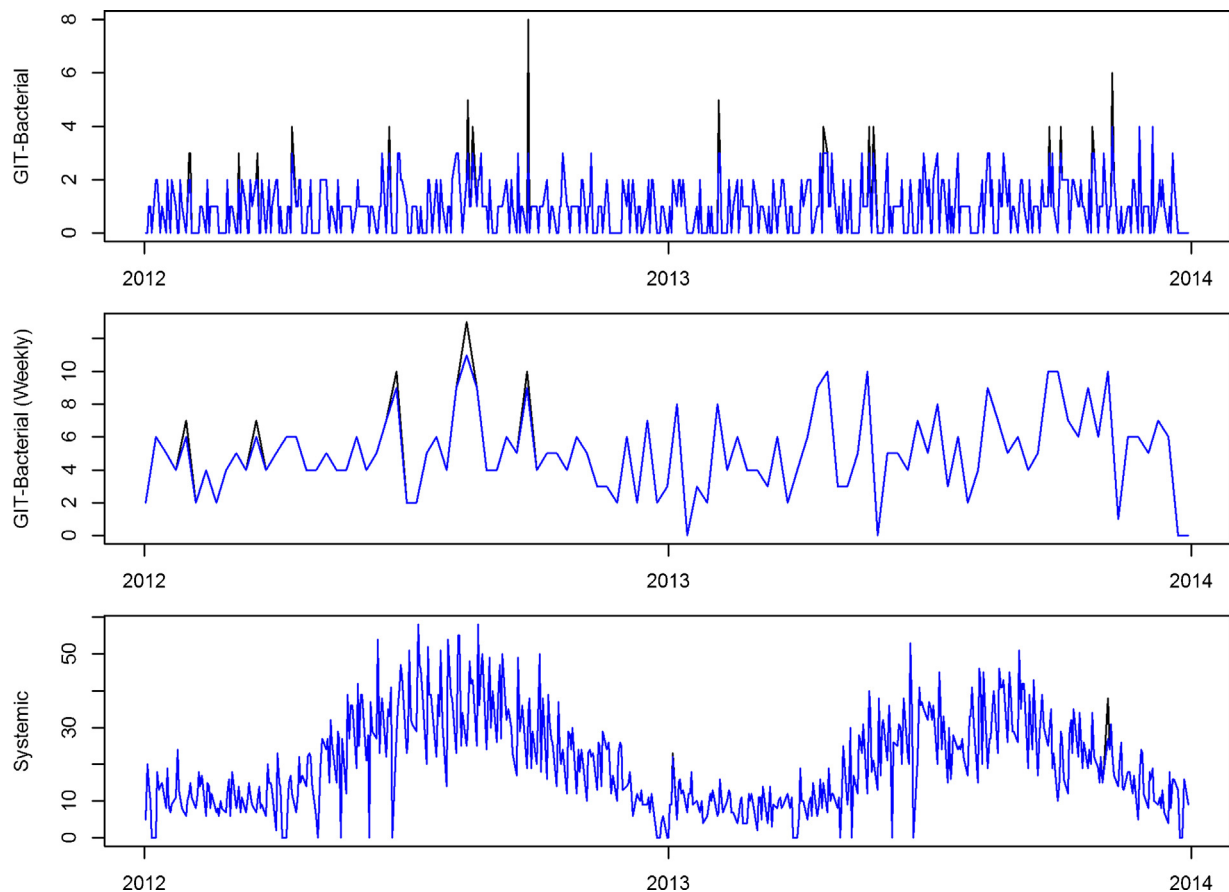


Fig. 5. Baselines constructed from the time series analysed, removing excessive noise and possible outbreak-signals. The black lines represent the observed number of events per day (or week), and it is superimposed by the blue line which represents the baseline constructed.

Two years were used as baseline, and detection was applied to the entire last year of data. Three detection limits were used (2.75, 3.0 and 3.25) and the middle limit was chosen for recording of the upper confidence limit (UCL). Alarms are to be recorded as a value between 0 and 3, for each algorithm employed individually, depending on the number of detection limits exceeded by observations. In the slot @alarms, therefore, a matrix of alarms for each syndrome and time point based on the results of the Holt-Winters algorithm is stored on the index 1 of the third dimension of the array (in R language, [,1]). Index 2 stores a similar matrix with the results of the EWMA algorithm, and index 3 the results of applying the Shewhart control charts. For the weekly data only the control charts were used.

2.9. Outputs

The figures presented in the results were generated with the function plot in R which, once the package is loaded, can recognize an object of the class *syndromic* and automatically produce plots similar to those shown. The user can choose which syndromes to plot, the window of plotting (in number of time points, always counted from the most recent), whether to plot the baseline, whether to plot the UCL for one of the algorithms employed, and which alarms to plot (if not specified the alarms for all algorithms employed are used, as shown in the results).

A single function (*syndromic_alarm*) can be used to automatically generate, from a *syndromic* object, an email in case alarms are present in the last time point. The user can also choose whether the email should contain a pdf attachment of the plots of the syndromes that generated alarms (or all syndromes), but using this

option requires setting an email of origin (information regarding the smtp Server of origin is needed). Emails with no attachment can be generated directly by the R interface. The user can also set an email recipient to receive a message attesting that the analyses were performed even when no alarms are detected, so that the system manager can be assured that the codes were executed successfully. The recipient lists in case of alarms or no alarm are set independently.

If the user wants to generate an html page with the summary of all analysis, a single function is available (*syndromic_page*) which plots a table summarizing the number of events for each syndrome, severity indicators for the alarm scores, and time series plots of all syndromes. The user can customize the window of time points in these plots, and the number of recent days (or weeks) to show in the summary tables. Examples are shown in the results.

3. Results

After the step of data preparation – transforming raw data into a syndromic object – the first results obtained from working with a data stream to be monitored, when using the *vetsyn* package, is the html page with summary statistics and exploratory analysis of the temporal effects in each of the syndromes found in the syndromic object.

Day-of-the-week effects were significant in all time series monitored. The absence of significant temporal patterns in the *GIT-Bacterial* time series was evident from the visual analysis of box-plots by month; the absence of significance of month, sine or cosine waves as predictors in both regression models tested (Poisson and negative binomial); and the failure of these same variables

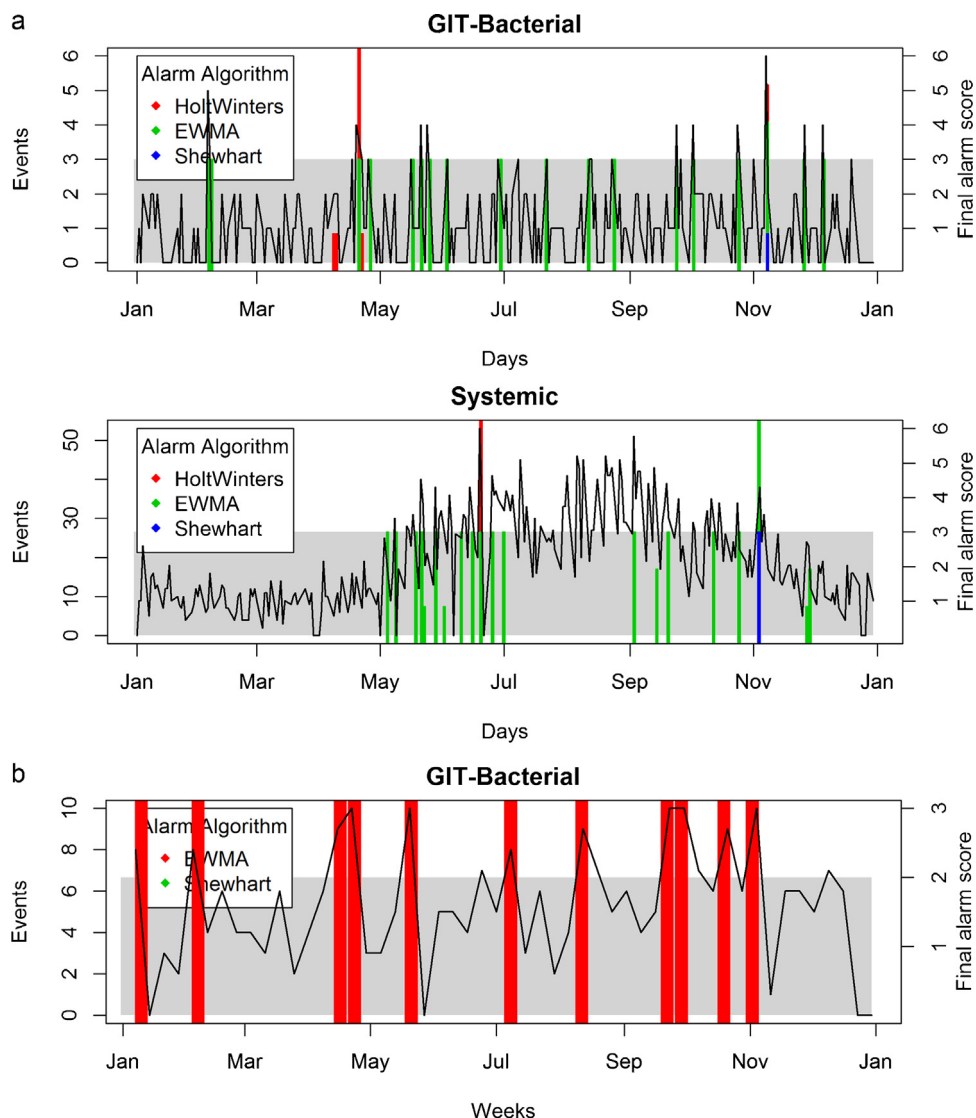


Fig. 6. Results of applying aberration detection to the evaluated time series of counts per day (6a) and per week (6b). Black lines show the observed number of events per day. Colored bars representing the alarm score, for each of three detection algorithms applied (scale on the secondary y-axis). The grey background reflects the alarm detection limit set by the user.

in improving fit as measured by the AIC. For the *Systemic* series, the negative binomial model showed better fit than the Poisson model. The models were significantly improved by the inclusion of a month variable, or a sine-cosine wave. The later was considered more appropriate due to the better fit, lower AIC, and because it provides a gradual curve of reduction or increase in the predicted number of events along the year (in contrast to the idea that the expected count changes from one day to the next as the calendar month changes). Within SVA's syndromic surveillance system, for a number of time series evaluated using the package methods (results not shown), the regression models showed lower AIC when temporal effects were modeled with a month variable, instead of a sine-cosine wave. However, when auto-regressive components from one to five days (one week in these data) were included in the regression models, both versions of each model showed similar AIC.

Based on these results, the observed time series were transformed into outbreak-free baselines, as shown in Fig. 5. For the *GIT-Bacterial* series, where no seasonal effects were observed, the baseline construction was performed using the non-parametric cleaning method available in the package (moving window of percentiles). For the daily series a moving window of 60 time points

(3 months, as weekends are not included) was set, and for the weekly series 12 weeks were used. For the *Systemic* series the GLM method was used, fitting a negative binomial regression model with day-of-week and sine-cosine waves as predictors, and removing observations above the 95% confidence interval.

Using the baselines constructed, the last year of data was subjected to outbreak signal detection as described in the methods. The results of applying the package functions using the parameters given in Table 2 are shown in Figs. 6 and 7, for the series without and with the injection of a simulated outbreak signal, respectively. Daily (Figs. 6 and 7a) and weekly (Figs. 6 and 7b) time series are shown. The figures are a result of applying the R function `plot` to the syndromic objects created. The plots show the time series of observed events in the main y-axis. Each alarm is represented by stacked bars on the day/week of the event. The height of the bars, indicated in the secondary y-axis, correspond to the number of detection limits exceeded for each detection algorithm. The maximum alarm score would be 9 for the daily syndromes, as we used three parallel algorithms, each working in three different detection limits; and 6 for the weekly series. The scale of the secondary y-axis is automatically adjusted to the maximum total alarm score found in the data.

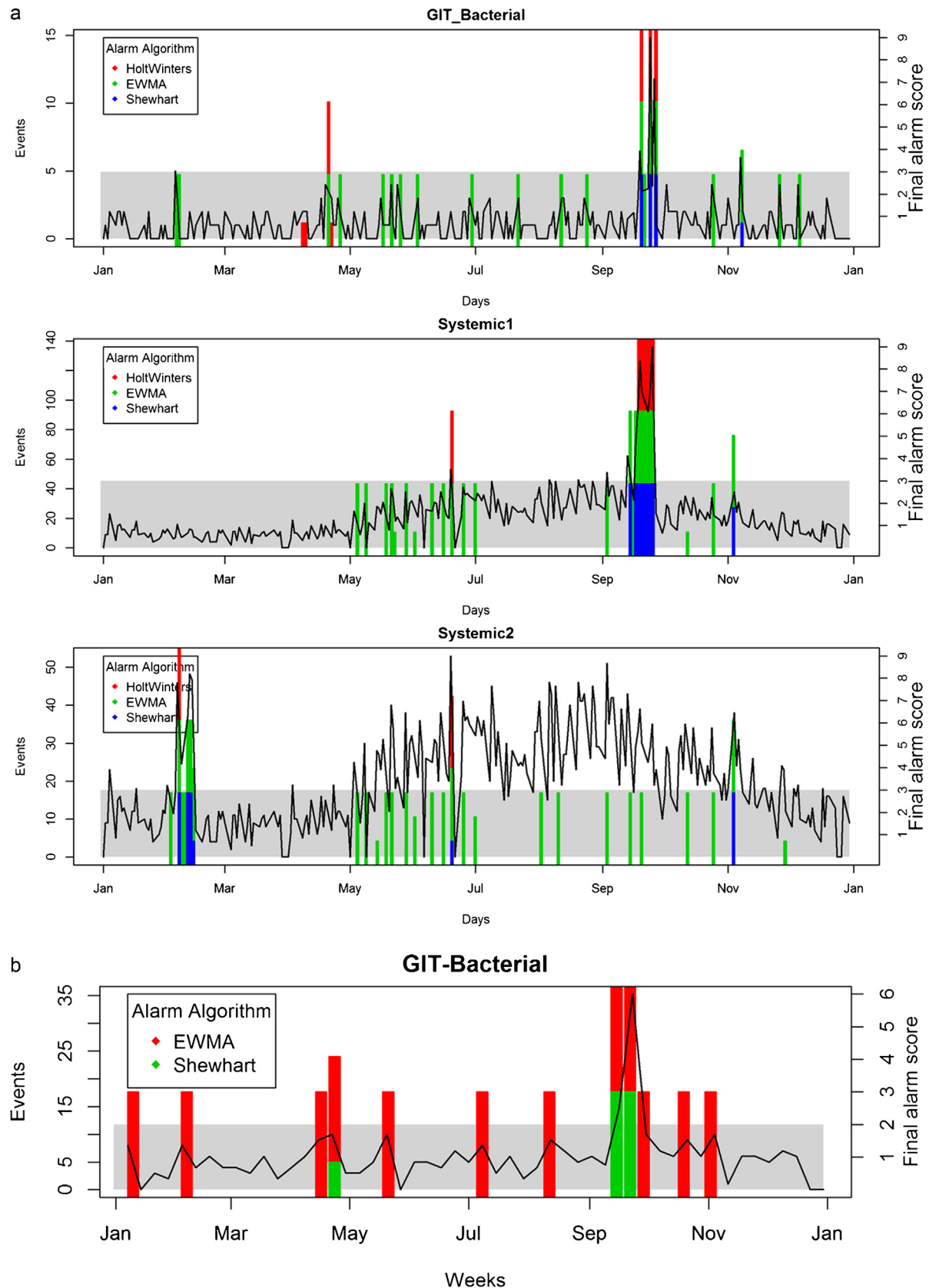


Fig. 7. Results of applying aberration detection to the evaluated time series, added of a one simulated outbreak per graph, when evaluated daily (6a) and weekly (6b). Black lines show the observed number of events per day. Colored bars representing the alarm score, for each of three detection algorithms applied (scale on the secondary y-axis). The grey background reflects the alarm detection limit set by the user.

Lab test requests - Dogs 2013-09-20

	Today		Previous Days History			
	Syndromic cases	Alarm Today	D-1	D-2	D-3	D-4
GIT_Bacterial	4		6	3	0	2
Systemic1	105		126	95	70	32

Fig. 8. Example heading of the html interface generated automatically by the vetsyn package.

In the plots, the grey background is intended to highlight the alarm detection limit set by the user, that is, the limit above which the total alarm score (the height of the colored bars in the graphs) would generate an alarm. For illustration, we have set this limit as 3 for the daily series (an alarm would be generated if for instance all algorithms had at least one detection limit exceeded, or a single algorithm had 3 detection limits exceeded) and 2 for the weekly series.

Assuming that there were no outbreaks in the year evaluated (2013), we would have observed 2 false alarms in each of the daily time series in that year (Fig. 6a). As it can be seen in the graph, the control charts performed really poorly in the series with high seasonal effects, despite the pre-processing using a regression methods with a sinusoidal wave. It has been our experience at SVA that for such series we should only trust the results of the Holt-Winters method, and therefore only one (false) alarm would have been detected in the *Systemic* time series. For the weekly series (Fig. 6b) the detection limit is clearly unfit, as many false alarms would have been generated. This particular syndrome used as an example here is monitored daily in our system, rather than weekly. But other syndromes monitored weekly in our system are monitored using five detection limits (2.75, 3, 3.25, 3.75, 4.25), and limits for the sum of alarm scores have been customized for each series individually to avoid excessive false alarms.

Fig. 7 shows detection of the outbreak signals injected. It is possible to see that the method available in the package by which an outbreak-free baseline is automatically kept was successful in avoiding that the outbreak signals injected would change the sensitivity of the detection algorithms. The days that triggered (false) alarms in the daily data (Fig. 7a) are still triggering alarms in the data with injected outbreaks, even after the outbreak days, showing that the training data for the algorithms was not contaminated. For the weekly data (Fig. 7b) it becomes clear that the detection limit should be set to take into account that both detection algorithms should signal an alarm, in order to differentiate false from true alarms. The first week with a detection score greater than 4 represented the first week in which outbreak signals were injected.

The html interface of the system that can be set up using the function `syndromic.page` contains the graphs as shown in Figs. 6 and 7. At the top of the page, a summary as shown in Fig. 8 would be generated. The page shown was generated 10 days after the start of the simulated outbreak (which grows on log-normal curve until a peak on day 15), and shows how the outbreak-signal has just been detected in the *GIT-Bacterial* series on the day before, and had been detected in the other time series for over 5 days

(injected outbreaks were proportional to the time series magnitude, and therefore greater in the *Systemic* series).

4. Discussion

We have presented an R package aimed at streamlining the process of converting an animal health data source into an operational syndromic surveillance system using a free software (R Core Team, 2012). It is assumed that users will handle data acquisition and update on their own, as well as classification of data into syndromes (when necessary), but all the statistical analyses steps have been programmed into a set of easy to apply R functions.

We have chosen to keep the package in GitHub in order to promote collaboration. We hope that other syndromic surveillance users will explore the package and suggest changes in cases when the framework is not found to attend their needs.

Although statistical methods for outbreak-signal detection were available in R (Höhle, 2007; Scrucca, 2004; Burkom et al., 2007), their application for the construction of a syndromic surveillance system relied on experienced users who could build a framework for data management, analysis, visualization and alarm generation. We have set up such a framework, and adapted these algorithms to perform analyses that attend the goals of an automated system for outbreak-system detection.

We aimed at developing a framework that was general enough to attend the needs of a great number of users. By streamlining the steps necessary to go from data to alarms, we have reduced the learning curve to develop such systems. Users are now guided through the steps they need to take, and the arguments for each function tell them exactly what decisions they need to make, based on the data available and the system needs. For the retrospective analysis of data, for instance, the function available in the package outputs a file with all R codes that would reproduce the analyses previously published in (Dórea et al., 2013b), and they can use that as a start to learn how to do some of these analyses in R.

While we have generalized those steps into a somewhat rigid framework, experienced users can still use the many arguments of each function to tailor the system, and achieve maximum performance for their individual data stream. Beginner users can start with the default settings of the functions, and improve the performance as they gain more knowledge into the system and their own data.

The use of object-oriented programming in R (S4 classes) (Chambers, 2014) presented two main advantages. The first was to facilitate data management, as all data needed for statistical analyses, as well as all outputs, are stored in the same object. The

second is that it allowed us to implement several validity checks. For instance, the counts of events are kept in a numerical-only matrix, while extensive date information about the time points are kept in a data frame in a separate slot. The correspondence between the rows in the matrix and the data frame is guaranteed by the internal validity checks. This reduces the chances of mistakes by users.

We have also associated the new S4 class created (syndromic) to specific behaviors of common R functions such as “print” and “plot”, facilitating the generation of epidemiologically relevant summaries and plots, both before and after analyses.

Our framework is mainly planned around the idea that a user would implement several outbreak-signal detection algorithms in parallel, and even set up more than one detection limit for each of those algorithms, a method which efficiency we have previously demonstrated (Dórea et al., 2013d). The framework can be as easily used with only one detection algorithm or single-valued detection limits. However, we have demonstrated here that the method based on multiple detection algorithms and multiple detection limits is highly customizable, and easy to tune in order to achieve a desired balance between sensitivity and specificity. For the two syndromes explored here, we have used the same settings for detection (the default settings in the package), but shown that the choice of which final alarm score (sum of alarm events as determined by all algorithms employed and all detection limits set) should produce an alarm must be set individually for each syndrome. This reflects the current framework of the full syndromic surveillance system implemented at SVA. The same settings for detection are applied to all syndromic groups followed, but the minimum alarm score that triggers an alert in the system has been chosen individually for each syndrome, with the aim of achieving high sensitivity, while not exceeding a false alarm rate of 3% (Reis et al., 2003).

We have demonstrated the results of algorithms for outbreak signal detection using simulated outbreaks. This is a common practice in the literature of syndromic surveillance (Buckeridge et al., 2005; Jackson et al., 2007; Wang et al., 2005; Hutwagner et al., 2005), due to the lack of documentation of real outbreaks in the syndromic data evaluated, and the difficulty of separating baseline from outbreak data when data do exist. However this limitation should be noted, as the efficacy of the algorithms depends on the shape, magnitude and duration of the true outbreak signals. This is the main reason to advocate the use of multiple aberration detection algorithms in parallel, as previously demonstrated (Dórea et al., 2013c).

Users wishing to develop and implement a VSS system should take into account the particularities of their data, as well as the expected shape of outbreak signals to be detected. Our goal was to demonstrate that the statistical process can be generalized, but the exact choice of detection limits, as well as the other parameters listed in Table 2, should be decided by each user for their own data. This decision should be based on finding a balance between sensitivity and specificity that attends the goals of their surveillance system. Such investigation can however be time consuming and depend on simulated data, therefore demanding technical and personal resources. Users should also pay special attention to the need to pre-process data before application of control charts. We have refrained from setting a default method for pre-processing of data in the package, as users should take that decision upon careful consideration of their data.

VSS systems developers should also consider the quality of the data available, as well as the many external factors that can affect the number of events recorded in their data, other than disease introduction (Dórea et al., 2011). While we have addressed here the statistical process to identify temporal aberrations in data, it is important to remember that this is just a first step. Epidemiologists should still review the statistical outputs, and take a decision

regarding whether these aberrations represent true outbreak signals.

To facilitate the process of reviewing outputs and data available, we have also provided users with a method to construct an output html page, and taken care of coding automated email alarms. At the moment only one output page can be created for each syndromic object. Codes to generate an interface that can aggregate all these individual pages through a menu, when multiple syndromic objects are created, are available upon contact with the first author.

The package was developed primarily for the use of veterinary data, but many of the statistical methods implemented will be useful for any syndromic data – human or animal. This first stage of development, however, has focused purely on temporal covariates for the statistical analyses. It is not possible, at the moment, to include in the regression models other covariates such as for instance age and sex. Users can set counts to be based on any level of aggregation (for instance individual animals or farms) but the hierarchical structure of the data cannot be accounted for (mixed models are not available in the package to deal, for instance, with animals and farms).

While the exploration of other sources of variation on data, besides temporal variables, may be addressed in the future, the next stages of development of the package are planned to focus on what we consider to be the two main current limitation of the package, for it to attend the more common needs of the veterinary syndromic surveillance community: the handling of proportion data; and the inclusion of spatial analysis.

The use of proportion data is common in veterinary medicine when monitoring, for instance, the number of condemnations from a total number of slaughtered animals per day. It can also refer to the proportion of units positive to certain screening tests. This is the subject of current research and future developments of the package.

After addressing statistical analysis of proportion data we also intend to explore the inclusion of spatial analysis. At the moment, although statistical analysis are not included with the package, if geographical information is available for the data users are welcome to contact the corresponding author for codes that allow maps to be added to a html interface for their VSS system.

We continue to develop the package based on the feedback of the VSS community, and users are encouraged to give feedback directly to the contact author, or creating discussions and suggesting changes using GitHub. Videos are available at the project webpage (www.sva.se, search for *vetsyn*) explaining how to collaborate through the code sharing platform GitHub.

5. Conclusions

R packages are available which support specific data analysis algorithms, but their application requires experienced users capable of incorporating these algorithms into the larger framework of a syndromic surveillance system. We presented a package in which various statistical analyses already tested for use in animal data streams are available, coupled with data management capabilities and output visualization. The main innovation offered by the package is ability to manage data streams, analyses, alarms and user interface in a continuous flow. We expect that this will facilitate the implementation of syndromic surveillance systems by veterinary epidemiologists and surveillance practitioners.

Acknowledgements

The project development was supported by the National Veterinary Institute of Sweden (SVA), and funded by the Swedish Board of Agriculture.

References

- Bradley, C.A., Rolka, H., Walker, D., Loonsk, J., 2005. BioSense: implementation of a national early event detection and situational awareness system. *Morbidity and Mortality Weekly Report* 54 Suppl., 11–19.
- Bravata, D.M., McDonald, K.M., Smith, W.M., Rydzak, C., Szeto, H., Buckeridge, D.L., Haberland, C., Owens, D.K., 2004. Systematic review: surveillance systems for early detection of bioterrorism-related diseases. *Ann. Intern. Med.* 140 (11), 910–922.
- Buckeridge, D.L., Burkom, H., Campbell, M., Hogan, W.R., Moore, A.W., 2005. Algorithms for rapid outbreak detection: a research synthesis. *J. Biomed. Inform.* 38 (2), 99–113.
- Burkom, H., Murphy, S., Shmueli, G., 2007. Automated time series forecasting for biosurveillance. *Stat. Med.* 26 (22), 4202–4218.
- Centers for Disease Control and Prevention (CDC), November 2013. Biosense: public health surveillance through collaboration. <http://www.cdc.gov/biosense/files/DHIS.BioSense%20Overview.244951.12.3.2013> (accessed 06.1.14.).
- Chambers, J., August 2006. How S4 methods work. <http://developer.r-project.org/howMethodsWork.pdf> Retrieved on January 21st, 2014.
- Dórea, F.C., Lindberg, A., McEwen, B.J., Sanchez, J., Revie, C.W., 2014. Syndromic surveillance using veterinary laboratory requests: a practical guide informed by experience with two systems. *Prev. Vet. Med.* 116, 313–324.
- Dórea, F.C., Muckle, C.A., Kelton, D., McClure, J., McEwen, B.J., McNab, W.B., Sanchez, J., Revie, C.W., 2013a. Exploratory analysis of methods for automated classification of laboratory test orders into syndromic groups in veterinary medicine. *PLoS One* 8 (3), e57334.
- Dórea, F.C., Revie, C.W., McEwen, B.J., McNab, W.B., Kelton, D., Sanchez, J., 2013b. Retrospective time series analysis of veterinary laboratory data preparing a historical baseline for cluster detection in syndromic surveillance. *Prev. Vet. Med.* 109, 219–227.
- Dórea, F.C., Revie, C.W., McEwen, B.J., McNab, W.B., Sanchez, J., 2013c. Syndromic surveillance using veterinary laboratory data: data pre-processing and algorithm performance evaluation. *J. R. Soc. Interface* 10 (83), 20130114.
- Dórea, F.C., Sanchez, J., McEwen, B.J., McNab, W.B., Revie, C.W., 2013d. Syndromic surveillance using veterinary laboratory data: algorithm combination and customization of alerts. *PLoS One* 8 (12), e82183.
- Dórea, F.C., Sanchez, J., Revie, C.W., 2011. Veterinary syndromic surveillance: current initiatives and potential for development. *Prev. Vet. Med.* 101 (1–2), 1–17.
- Dupuy, C., Bronner, A., Watson, E., Wuyckhuise-Sjouke, L., Reist, M., Fouillet, A., Calavasa, D., Hendrikx, P., Perrin, J.-B., 2013. Inventory of veterinary syndromic surveillance initiatives in Europe (Triple-s project): current situation and perspectives. *Prev. Vet. Med.* 111, 220–229.
- Elbert, Y., Burkom, H.S., 2009. Development and evaluation of a data-adaptive alerting algorithm for univariate temporal biosurveillance data. *Stat. Med.* 28, 3226–3248.
- Fouillet, A., Medina, S., Viso, A., Caserio-Schönemann, C., Medeiros, H., 2013. Guidelines for implementing syndromic surveillance in Europe and proposal for a European syndromic surveillance strategy. *Eur. J. Public Health* 23 (suppl. 1) <http://eurpub.oxfordjournals.org/content/23/suppl.1/ckt126.117>.
- Höhle, M., 2007. Surveillance: an R package for the surveillance of infectious diseases. *Comput. Stat.* 22 (4), 571–582.
- Hutwagner, L., Browne, T., Seeman, G.M., Fleischauer, A.T., 2005. Comparing aberration detection methods with simulated data. *Emerg. Infect. Dis.* 11, 314–316.
- Jackson, M.L., Baer, A., Painter, I., Duchin, J.A., 2007. A simulation study comparing aberration detection algorithms for syndromic surveillance. *BMC Med. Inf. Decis. Making* 7, 6.
- Lotze, T., Murphy, S., Shmueli, G., 2008. Implementation and comparison of preprocessing methods for biosurveillance data. *Adv. Dis. Surveill.* 6 (1), 1–20.
- R Core Team, 2012. R: A Language and Environment for Statistical Computing. R Foundation for Statistical Computing, Vienna, Austria, ISBN 3-900051-07-0. <http://www.R-project.org>.
- Reis, B.Y., Pagano, M., Mandl, K.D., 2003. Using temporal context to improve biosurveillance. *Proc. Natl. Acad. Sci. U. S. A.* 100 (4), 1961–1965.
- Scrucca, L., 2004. qcc: an r package for quality control charting and statistical process control. *R News* 4/1, 11–17. <http://CRAN.R-project.org/doc/Rnews/>.
- Tsui, F.C., Wagner, M.M., Dato, V., Chang, C.C., 2001. Value of ICD-9 coded chief complaints for detection of epidemics. *AMIA Annu. Symp. Proc.* 1, 711–715.
- Wang, L., Ramoni, M.F., Mandl, K.D., Sebastiani, P., 2005. Factors affecting automated syndromic surveillance. *Artif. Intell. Med.* 34, 269–278.
- Woodall, W.H., 2006. Use of control charts in healthcare and public-health surveillance. *J. Qual. Technol.* 38, 89–104.
- Xie, Y., 2014. knitr: A general-purpose package for dynamic report generation in R. R package version 1.6. <http://yihui.name/knitr/>.
- Zeng D., Chen H., Lynch C., Eidson M., Gotham I., 2005. Infectious Disease Informatics and Outbreak Detection. Knowledge Management and Data Mining in Biomedicine. Springer ID., 1226.